

Clase 07 - Tipos de Join

Sesión 1

Tipos de JOIN

1. Contexto

Una empresa de servicios tecnológicos tiene:

- departamentos;
- empleados;
- empleados asignados a departamentos;
- empleados que pueden tener un supervisor dentro de la misma tabla.

Necesitamos entender cómo funcionan los principales tipos de JOIN en Oracle SQL utilizando únicamente **dos tablas principales**:

```
EMP_DEPARTAMENTOS  
EMP_EMPLEADOS
```

Con estas dos tablas vamos a ver ejemplos de:

- INNER JOIN ;
- LEFT JOIN o LEFT OUTER JOIN ;
- RIGHT JOIN o RIGHT OUTER JOIN ;
- FULL JOIN o FULL OUTER JOIN ;
- CROSS JOIN ;
- SELF JOIN .

2. Creación del usuario de prácticas

2.1. Conectarse como usuario administrador

Antes de crear las tablas, hay que conectarse como usuario Admin y crear el usuario y la conexión.

2.2. Crear usuario para esta práctica

Ejecutar el siguiente script desde una conexión administrativa:

```
CREATE USER join_lab IDENTIFIED BY JoinLab_123;
```

```
GRANT CREATE SESSION TO join_lab;  
GRANT CREATE TABLE TO join_lab;  
GRANT CREATE VIEW TO join_lab;  
GRANT CREATE SEQUENCE TO join_lab;  
GRANT CREATE PROCEDURE TO join_lab;  
GRANT CREATE TRIGGER TO join_lab;  
  
ALTER USER join_lab QUOTA UNLIMITED ON USERS;
```

2.3. Crear conexión en Oracle SQL Developer

Crear una nueva conexión con estos datos orientativos:

```
Name: JOIN_LAB  
Usuario: join_lab  
Contraseña: JoinLab_123  
Hostname: localhost  
Puerto: 1521  
Servicio: FREEPDB1
```

Después de crear la conexión, pulsar Probar. Si la conexión es correcta, pulsar **Connect**.

3. Definición de **JOIN**



Un **JOIN** es una operación SQL que permite **combinar filas de dos o más tablas** usando una condición de relación entre ellas. Normalmente esa condición compara una **clave primaria** de una tabla con una **clave foránea** de otra tabla.

Sintaxis:

```
SELECT columnas  
FROM tabla_1  
JOIN tabla_2  
ON tabla_1.columna = tabla_2.columna;
```

En esta práctica:

- **EMP_DEPARTAMENTOS.ID_DEPARTAMENTO** será la clave primaria de departamentos.
- **EMP_EMPLEADOS.ID_DEPARTAMENTO** será la clave foránea que apunta a departamentos.

4. Creación de las tablas

4.1. Limpieza previa opcional

Si ya existen las tablas de una ejecución anterior, se pueden borrar con este script:

```
DROP TABLE emp_empleados CASCADE CONSTRAINTS;  
DROP TABLE emp_departamentos CASCADE CONSTRAINTS;
```

Si las tablas no existen, Oracle mostrará un error indicando que la tabla no existe. Ese error se puede ignorar en esta fase.

4.2. Tabla **EMP_DEPARTAMENTOS**

Esta tabla almacena los departamentos de la empresa.

```
CREATE TABLE emp_departamentos (  
    id_departamento NUMBER,  
    nombre            VARCHAR2(80) NOT NULL,  
    sede              VARCHAR2(80) NOT NULL,  
    presupuesto       NUMBER(10,2) NOT NULL,  
    activo             CHAR(1)      DEFAULT 'S' NOT NULL,  
  
    CONSTRAINT pk_emp_departamentos  
        PRIMARY KEY (id_departamento),  
  
    CONSTRAINT uk_emp_departamentos_nombre  
        UNIQUE (nombre),  
  
    CONSTRAINT ck_emp_departamentos_presupuesto  
        CHECK (presupuesto >= 0),  
  
    CONSTRAINT ck_emp_departamentos_activo  
        CHECK (activo IN ('S', 'N'))  
);
```

Restricciones de la tabla

Restricción	Función
PRIMARY KEY	Identifica de forma única cada departamento.
UNIQUE	Evita que dos departamentos tengan el mismo nombre.
NOT NULL	Obliga a registrar datos importantes.
CHECK presupuesto >= 0	Evita presupuestos negativos.
CHECK activo IN ('S', 'N')	Solo permite indicar si el departamento está activo o no.

4.3. Tabla **EMP_EMPLEADOS**

Esta tabla almacena los empleados de la empresa.

```
CREATE TABLE emp_empleados (  
    id_empleado      NUMBER,  
    nombre            VARCHAR2(80) NOT NULL,
```

```

email            VARCHAR2(120) NOT NULL,
puesto           VARCHAR2(80)  NOT NULL,
salario          NUMBER(10,2)  NOT NULL,
fecha_alta       DATE          DEFAULT SYSDATE NOT NULL,
id_departamento NUMBER,
id_supervisor     NUMBER,
estado           VARCHAR2(20)  DEFAULT 'ACTIVO' NOT NULL,

CONSTRAINT pk_emp_empleados
    PRIMARY KEY (id_empleado),

CONSTRAINT uk_emp_empleados_email
    UNIQUE (email),

CONSTRAINT fk_emp_empleados_departamentos
    FOREIGN KEY (id_departamento)
    REFERENCES emp_departamentos (id_departamento),

CONSTRAINT fk_emp_empleados_supervisor
    FOREIGN KEY (id_supervisor)
    REFERENCES emp_empleados (id_empleado),

CONSTRAINT ck_emp_empleados_salario
    CHECK (salario > 0),

CONSTRAINT ck_emp_empleados_estado
    CHECK (estado IN ('ACTIVO', 'BAJA', 'VACACIONES')),

CONSTRAINT ck_emp_empleados_no_autosupervisor
    CHECK (id_supervisor IS NULL OR id_supervisor <> id_empleado)
);

```

Restricciones de la tabla

Restricción	Función
PRIMARY KEY	Identifica de forma única cada empleado.
UNIQUE	Evita emails repetidos.
FOREIGN KEY id_departamento	Relaciona empleados con departamentos.
FOREIGN KEY id_supervisor	Relaciona un empleado con otro empleado que actúa como supervisor.
CHECK salario > 0	Evita salarios negativos o cero.
CHECK estado IN (...)	Solo permite estados válidos.
CHECK id_supervisor <> id_empleado	Evita que un empleado sea su propio supervisor.

5. Inserción de registros

5.1. Insertar 20 departamentos

```
INSERT INTO emp_departamentos VALUES (1, 'Dirección General', 'Madrid', 250000, 'S');
INSERT INTO emp_departamentos VALUES (2, 'Recursos Humanos', 'Madrid', 90000, 'S');
INSERT INTO emp_departamentos VALUES (3, 'Desarrollo Software', 'Valencia', 180000, 'S');
INSERT INTO emp_departamentos VALUES (4, 'Soporte Técnico', 'Sevilla', 120000, 'S');
INSERT INTO emp_departamentos VALUES (5, 'Ciberseguridad', 'Madrid', 160000, 'S');
INSERT INTO emp_departamentos VALUES (6, 'Marketing Digital', 'Barcelona', 110000, 'S');
INSERT INTO emp_departamentos VALUES (7, 'Ventas Corporativas', 'Barcelona', 150000, 'S');
INSERT INTO emp_departamentos VALUES (8, 'Finanzas', 'Madrid', 130000, 'S');
INSERT INTO emp_departamentos VALUES (9, 'Legal', 'Madrid', 80000, 'S');
INSERT INTO emp_departamentos VALUES (10, 'Calidad', 'Valencia', 70000, 'S');
INSERT INTO emp_departamentos VALUES (11, 'Innovación', 'Bilbao', 140000, 'S');
INSERT INTO emp_departamentos VALUES (12, 'Formación Interna', 'Sevilla', 60000, 'S');
INSERT INTO emp_departamentos VALUES (13, 'Atención al Cliente', 'Valencia', 100000, 'S');
INSERT INTO emp_departamentos VALUES (14, 'Infraestructura Cloud', 'Madrid', 200000, 'S');
INSERT INTO emp_departamentos VALUES (15, 'Datos y Analítica', 'Barcelona', 190000, 'S');
INSERT INTO emp_departamentos VALUES (16, 'Compras', 'Zaragoza', 65000, 'S');
INSERT INTO emp_departamentos VALUES (17, 'Logística', 'Zaragoza', 85000, 'S');
INSERT INTO emp_departamentos VALUES (18, 'Producto', 'Bilbao', 125000, 'S');
INSERT INTO emp_departamentos VALUES (19, 'Expansión Internacional', 'Madrid', 175000, 'N');
INSERT INTO emp_departamentos VALUES (20, 'Laboratorio IA', 'Málaga', 220000, 'S');

COMMIT;
```

5.2. Insertar 20 empleados

Algunos empleados tienen departamento y otros no. Esto se hace de forma intencionada para poder ver bien la diferencia entre los distintos tipos de `JOIN`.

```
INSERT INTO emp_empleados VALUES (1, 'Laura Gómez', 'laura.gomez@emp
resa.com', 'Directora General', 65000, DATE '2023-01-10',
1, NULL, 'ACTIVO');
INSERT INTO emp_empleados VALUES (2, 'Carlos Ruiz', 'carlos.ruiz@emp
resa.com', 'Responsable RRHH', 42000, DATE '2023-02-01',
2, 1, 'ACTIVO');
INSERT INTO emp_empleados VALUES (3, 'Marta León', 'marta.leon@empr
esa.com', 'Jefa Desarrollo', 52000, DATE '2023-02-15',
3, 1, 'ACTIVO');
INSERT INTO emp_empleados VALUES (4, 'Daniel Torres', 'daniel.torres@e
mpresa.com', 'Técnico Soporte', 28000, DATE '2023-03-05',
4, 3, 'ACTIVO');
INSERT INTO emp_empleados VALUES (5, 'Ana Molina', 'ana.molina@empr
esa.com', 'Analista Ciberseguridad', 39000, DATE '2023-03-18',
5, 3, 'ACTIVO');
INSERT INTO emp_empleados VALUES (6, 'Pedro Sánchez', 'pedro.sanchez@e
mpresa.com', 'Especialista Marketing', 31000, DATE '2023-04-02',
6, 1, 'ACTIVO');
INSERT INTO emp_empleados VALUES (7, 'Sofía Martín', 'sofia.martin@em
presa.com', 'Ejecutiva Ventas', 36000, DATE '2023-04-20',
7, 1, 'ACTIVO');
INSERT INTO emp_empleados VALUES (8, 'Javier Ortega', 'javier.ortega@e
mpresa.com', 'Contable', 30000, DATE '2023-05-10',
8, 1, 'ACTIVO');
INSERT INTO emp_empleados VALUES (9, 'Lucía Herrera', 'lucia.herrera@e
mpresa.com', 'Abogada Corporativa', 41000, DATE '2023-05-22',
9, 1, 'ACTIVO');
INSERT INTO emp_empleados VALUES (10, 'Miguel Castro', 'miguel.castro@e
mpresa.com', 'Técnico Calidad', 29000, DATE '2023-06-01',
10, 3, 'ACTIVO');
INSERT INTO emp_empleados VALUES (11, 'Elena Navarro', 'elena.navarro@e
mpresa.com', 'Consultora Innovación', 45000, DATE '2023-06-12',
11, 1, 'ACTIVO');
INSERT INTO emp_empleados VALUES (12, 'Raúl Jiménez', 'raul.jimenez@em
presa.com', 'Formador Interno', 32000, DATE '2023-07-03',
12, 2, 'ACTIVO');
INSERT INTO emp_empleados VALUES (13, 'Paula Romero', 'paula.romero@em
presa.com', 'Agente Atención Cliente', 26000, DATE '2023-07-20',
13, 2, 'ACTIVO');
INSERT INTO emp_empleados VALUES (14, 'Iván Delgado', 'ivan.delgado@em
presa.com', 'Administrador Cloud', 47000, DATE '2023-08-08',
14, 3, 'ACTIVO');
INSERT INTO emp_empleados VALUES (15, 'Nuria Vega', 'nuria.vega@empr
```

```

esa.com',      'Analista de Datos',      44000, DATE '2023-08-26',
15, 3,      'VACACIONES');
INSERT INTO emp_empleados VALUES (16, 'Óscar Medina',      'oscar.medina@em
presa.com',      'Técnico de Compras',      27000, DATE '2023-09-11',
16, 8,      'ACTIVO');
INSERT INTO emp_empleados VALUES (17, 'Teresa Campos',      'teresa.campos@e
mpresa.com',      'Coordinadora Logística',      35000, DATE '2023-09-29',
17, 8,      'ACTIVO');
INSERT INTO emp_empleados VALUES (18, 'Andrés Prieto',      'andres.prieto@e
mpresa.com',      'Product Owner',      50000, DATE '2023-10-15',
18, 1,      'ACTIVO');
INSERT INTO emp_empleados VALUES (19, 'Beatriz Salas',      'beatriz.salas@e
mpresa.com',      'Consultora Externa',      38000, DATE '2023-11-05',
NULL, 1,      'ACTIVO');
INSERT INTO emp_empleados VALUES (20, 'Hugo Rivas',      'hugo.rivas@empr
esa.com',      'Becario Proyecto Especial',      18000, DATE '2023-11-20',
NULL, 3,      'ACTIVO');

COMMIT;

```

6. Comprobación inicial de datos

Antes de hacer **JOIN**, conviene revisar qué hay en cada tabla.

```

SELECT *
FROM emp_departamentos;

```

```

SELECT *
FROM emp_empleados;

```

También se puede comprobar cuántos registros hay en cada tabla:

```

SELECT COUNT(*) AS total_departamentos
FROM emp_departamentos;

```

```

SELECT COUNT(*) AS total_empleados
FROM emp_empleados;

```

Resultado esperado:

```

TOTAL_DEPARTAMENTOS: 20
TOTAL_EMPLEADOS:    20

```

7. Tipos de **JOIN**

7.1. INNER JOIN

Definición



INNER JOIN devuelve solo las filas que tienen coincidencia en ambas tablas. En este caso, mostrará únicamente los empleados que tienen un departamento válido asignado.

Los empleados sin departamento no aparecerán.

Ejemplo

```
SELECT
  e.id_empleado,
  e.nombre AS empleado,
  e.puesto,
  d.nombre AS departamento,
  d.sede
FROM emp_empleados e
INNER JOIN emp_departamentos d
  ON e.id_departamento = d.id_departamento
ORDER BY e.id_empleado;
```

Qué se debe observar

Este **JOIN** solo muestra empleados cuyo **ID_DEPARTAMENTO** existe en la tabla **EMP_DEPARTAMENTOS**.

Los empleados **Beatriz Salas** y **Hugo Rivas** no aparecerán porque tienen **ID_DEPARTAMENTO** en **NULL**.

7.2. LEFT JOIN o LEFT OUTER JOIN

Definición

LEFT JOIN devuelve todas las filas de la tabla de la izquierda y, cuando encuentra coincidencia, añade los datos de la tabla de la derecha.

Si no hay coincidencia, las columnas de la tabla derecha aparecen como **NULL**.

En este caso, si ponemos **EMP_EMPLEADOS** a la izquierda, aparecerán todos los empleados, incluso los que no tienen departamento.

Ejemplo

```
SELECT
  e.id_empleado,
  e.nombre AS empleado,
  e.puesto,
  e.id_departamento,
  d.nombre AS departamento,
  d.sede
```

```
FROM emp_empleados e
LEFT JOIN emp_departamentos d
    ON e.id_departamento = d.id_departamento
ORDER BY e.id_employado;
```

Qué se debe observar

Aparecen los 20 empleados. Los empleados sin departamento aparecen con los datos del departamento en `NULL`.

7.3. `RIGHT JOIN` o `RIGHT OUTER JOIN`

Definición



`RIGHT JOIN` devuelve todas las filas de la tabla de la derecha y, cuando encuentra coincidencia, añade los datos de la tabla de la izquierda. Si no hay coincidencia, las columnas de la tabla izquierda aparecen como `NULL`.

En este caso, si ponemos `EMP_DEPARTAMENTOS` a la derecha, aparecerán todos los departamentos, incluso los que no tienen empleados asignados.

Ejemplo

```
SELECT
    e.id_employado,
    e.nombre AS empleado,
    d.id_departamento,
    d.nombre AS departamento,
    d.sede
FROM emp_empleados e
RIGHT JOIN emp_departamentos d
    ON e.id_departamento = d.id_departamento
ORDER BY d.id_departamento, e.id_employado;
```

Qué se debe observar

Aparecen todos los departamentos. Los departamentos sin empleados asignados aparecerán con las columnas del empleado en `NULL`. Por ejemplo, el departamento `Laboratorio IA` no tiene empleados asignados en los registros insertados.

7.4. `FULL JOIN` o `FULL OUTER JOIN`

Definición



FULL JOIN devuelve:

- las filas que coinciden en ambas tablas;
- las filas de la tabla izquierda que no tienen coincidencia;
- las filas de la tabla derecha que no tienen coincidencia.

Es como combinar el resultado de un **LEFT JOIN** y un **RIGHT JOIN**.

Ejemplo

```
SELECT
  e.id_employado,
  e.nombre AS empleado,
  e.puesto,
  d.id_departamento,
  d.nombre AS departamento,
  d.sede
FROM emp_empleados e
FULL OUTER JOIN emp_departamentos d
  ON e.id_departamento = d.id_departamento
ORDER BY d.id_departamento, e.id_employado;
```

Este resultado incluye:

- empleados con departamento;
- empleados sin departamento;
- departamentos sin empleados.

Es útil cuando se quiere revisar información incompleta o detectar datos pendientes de asignación.

7.5. **CROSS JOIN**

Definición



CROSS JOIN devuelve todas las combinaciones posibles entre las filas de dos tablas. No utiliza condición **ON**.

Si una tabla tiene 20 registros y la otra tiene 20 registros, el resultado tendrá:

$20 \times 20 = 400$ filas

Ejemplo

```
SELECT
    e.nombre AS empleado,
    d.nombre AS posible_departamento
FROM emp_empleados e
CROSS JOIN emp_departamentos d
ORDER BY e.nombre, d.nombre;
```

Qué se debe observar

Cada empleado se combina con todos los departamentos. Este tipo de **JOIN** debe usarse con cuidado porque puede generar resultados muy grandes.

Comprobación del número de combinaciones

```
SELECT COUNT(*) AS total_combinaciones
FROM emp_empleados e
CROSS JOIN emp_departamentos d;
```

Resultado esperado:

```
TOTAL_COMBINACIONES: 400
```

7.6. SELF JOIN

Definición



SELF JOIN es una consulta donde una tabla se une consigo misma. Se usa cuando una tabla tiene una relación interna.

En esta práctica, la tabla **EMP_EMPLEADOS** tiene la columna **ID_SUPERVISOR**, que apunta a otro empleado de la misma tabla.

Por eso se puede usar **SELF JOIN** para mostrar cada empleado junto a su supervisor.

Ejemplo

```
SELECT
    e.id_empleado,
    e.nombre AS empleado,
    e.puesto AS puesto_empleado,
    s.nombre AS supervisor,
    s.puesto AS puesto_supervisor
FROM emp_empleados e
LEFT JOIN emp_empleados s
```

```
ON e.id_supervisor = s.id_empleado
ORDER BY e.id_empleado;
```

Qué se debe observar

La misma tabla se usa dos veces:

```
e = empleado
s = supervisor
```

La tabla `EMP_EMPLEADOS` se comporta como si fueran dos tablas distintas gracias a los alias.

El empleado `Laura Gómez` no tiene supervisor, por eso aparece con el supervisor en `NULL`.

8. Comparación rápida de los tipos de JOIN

Tipo de JOIN	Qué devuelve
<code>INNER JOIN</code>	Solo coincidencias entre ambas tablas.
<code>LEFT JOIN</code>	Todo lo de la izquierda y las coincidencias de la derecha.
<code>RIGHT JOIN</code>	Todo lo de la derecha y las coincidencias de la izquierda.
<code>FULL JOIN</code>	Coincidencias y no coincidencias de ambas tablas.
<code>CROSS JOIN</code>	Todas las combinaciones posibles.
<code>SELF JOIN</code>	Una tabla relacionada consigo misma.

9. Ejercicios

9.1. Escenario de trabajo

En esta práctica se trabajará con el caso de una **cadena de tiendas**.

Se utilizarán tres tablas:

- `TIENDAS`
- `EMPLEADOS`
- `VENTAS`

Con ellas se podrán practicar distintos tipos de `JOIN` :

- `INNER JOIN`
- `LEFT JOIN`
- `RIGHT JOIN`
- `FULL OUTER JOIN`
- `CROSS JOIN`
- `SELF JOIN`

9.2. Crear usuario de práctica

```
CREATE USER JOIN_STORE IDENTIFIED BY JoinStore_123;

GRANT CREATE SESSION TO JOIN_STORE;
GRANT CREATE TABLE TO JOIN_STORE;
GRANT CREATE VIEW TO JOIN_STORE;
GRANT CREATE SEQUENCE TO JOIN_STORE;
GRANT CREATE PROCEDURE TO JOIN_STORE;
GRANT CREATE TRIGGER TO JOIN_STORE;

ALTER USER JOIN_STORE QUOTA UNLIMITED ON USERS;
```

9.3. Crear la conexión

1. Abrir **SQL Developer**.
2. Pulsar en **New Connection**.
3. Completar los datos:
 - **Connection Name:** JOIN_STORE
 - **Username:** JOIN_STORE
 - **Password:** JoinStore_123
4. Configurar los datos de conexión según la base de datos instalada.
5. Pulsar **Test (prueba)**.
6. Si la prueba es correcta, pulsar **Connect**.

9.4. Crear las tablas

Tabla 1: TIENDAS

```
CREATE TABLE tiendas (
    id_tienda          NUMBER PRIMARY KEY,
    nombre_tienda      VARCHAR2(100) NOT NULL,
    ciudad             VARCHAR2(50) NOT NULL,
    tipo_tienda        VARCHAR2(20) NOT NULL,
    telefono            VARCHAR2(20) UNIQUE,
    CONSTRAINT chk_tipo_tienda
        CHECK (tipo_tienda IN ('Centro', 'Barrio', 'Outlet'))
);
```

Tabla 2: EMPLEADOS

```
CREATE TABLE empleados (
    id_empleado        NUMBER PRIMARY KEY,
```

```

nombre          VARCHAR2(100) NOT NULL,
cargo           VARCHAR2(50) NOT NULL,
salario         NUMBER(8,2) NOT NULL,
id_tienda       NUMBER,
id_supervisor   NUMBER,
CONSTRAINT chk_salario
    CHECK (salario > 0),
CONSTRAINT fk_empleados_tiendas
    FOREIGN KEY (id_tienda)
    REFERENCES tiendas(id_tienda),
CONSTRAINT fk_empleados_supervisor
    FOREIGN KEY (id_supervisor)
    REFERENCES empleados(id_empleado)
);

```

Tabla 3: VENTAS

```

CREATE TABLE ventas (
    id_venta      NUMBER PRIMARY KEY,
    fecha_venta   DATE NOT NULL,
    monto         NUMBER(10,2) NOT NULL,
    metodo_pago   VARCHAR2(20) NOT NULL,
    id_empleado   NUMBER NOT NULL,
    id_tienda     NUMBER NOT NULL,
    CONSTRAINT chk_monto
        CHECK (monto >= 0),
    CONSTRAINT chk_metodo_pago
        CHECK (metodo_pago IN ('Tarjeta', 'Efectivo', 'Transferencia')),
    CONSTRAINT fk_ventas_empleados
        FOREIGN KEY (id_empleado)
        REFERENCES empleados(id_empleado),
    CONSTRAINT fk_ventas_tiendas
        FOREIGN KEY (id_tienda)
        REFERENCES tiendas(id_tienda)
);

```

9.5. Insertar registros

Registros en TIENDAS

```

INSERT INTO tiendas VALUES (1, 'Tienda Sol', 'Madrid', 'Centro', '91000000
1');
INSERT INTO tiendas VALUES (2, 'Tienda Norte', 'Madrid', 'Barrio', '910000
002');
INSERT INTO tiendas VALUES (3, 'Tienda Sur', 'Sevilla', 'Centro', '9100000
03');

```

```

INSERT INTO tiendas VALUES (4, 'Tienda Este', 'Valencia', 'Outlet', '910000004');
INSERT INTO tiendas VALUES (5, 'Tienda Oeste', 'Bilbao', 'Barrio', '910000005');
INSERT INTO tiendas VALUES (6, 'Tienda Mar', 'Málaga', 'Centro', '910000006');
INSERT INTO tiendas VALUES (7, 'Tienda Plaza', 'Zaragoza', 'Barrio', '910000007');
INSERT INTO tiendas VALUES (8, 'Tienda Centro', 'Barcelona', 'Centro', '910000008');
INSERT INTO tiendas VALUES (9, 'Tienda Río', 'Murcia', 'Outlet', '910000009');
INSERT INTO tiendas VALUES (10, 'Tienda Azul', 'Alicante', 'Centro', '910000010');

```

Registros en EMPLEADOS

```

INSERT INTO empleados VALUES (101, 'María López', 'Supervisora', 2200, 1, NULL);
INSERT INTO empleados VALUES (102, 'Carlos Ruiz', 'Supervisor', 2250, 2, NULL);
INSERT INTO empleados VALUES (103, 'Lucía Gómez', 'Vendedora', 1450, 1, 101);
INSERT INTO empleados VALUES (104, 'Pedro Sánchez', 'Vendedor', 1500, 1, 101);
INSERT INTO empleados VALUES (105, 'Ana Torres', 'Cajera', 1400, 2, 102);
INSERT INTO empleados VALUES (106, 'Jorge Martín', 'Vendedor', 1480, 2, 102);
INSERT INTO empleados VALUES (107, 'Elena Navarro', 'Vendedora', 1520, 3, 101);
INSERT INTO empleados VALUES (108, 'Sofía Díaz', 'Cajera', 1410, 3, 101);
INSERT INTO empleados VALUES (109, 'David Romero', 'Vendedor', 1490, 4, 102);
INSERT INTO empleados VALUES (110, 'Paula Herrera', 'Encargada', 1800, 5, 102);
INSERT INTO empleados VALUES (111, 'Miguel Castro', 'Vendedor', 1475, 5, 101);
INSERT INTO empleados VALUES (112, 'Laura Peña', 'Vendedora', 1510, 6, 110);
INSERT INTO empleados VALUES (113, 'Raúl Ortega', 'Cajero', 1390, 7, 110);
INSERT INTO empleados VALUES (114, 'Nuria Vidal', 'Vendedora', 1505, 8, 101);
INSERT INTO empleados VALUES (115, 'Irene León', 'Vendedora', 1495, NULL, 102);

```

Observa que:

- la tienda **9** y la tienda **10** no tienen empleados asignados;
- el empleado **115** no tiene tienda asignada;
- esto permitirá practicar mejor los distintos tipos de **JOIN**.

Registros en VENTAS

```
INSERT INTO ventas VALUES (1001, DATE '2026-06-01', 120.50, 'Tarjeta', 103, 1);
INSERT INTO ventas VALUES (1002, DATE '2026-06-01', 85.00, 'Efectivo', 104, 1);
INSERT INTO ventas VALUES (1003, DATE '2026-06-02', 210.75, 'Tarjeta', 105, 2);
INSERT INTO ventas VALUES (1004, DATE '2026-06-02', 99.99, 'Transferencia', 106, 2);
INSERT INTO ventas VALUES (1005, DATE '2026-06-03', 150.00, 'Tarjeta', 107, 3);
INSERT INTO ventas VALUES (1006, DATE '2026-06-03', 60.25, 'Efectivo', 108, 3);
INSERT INTO ventas VALUES (1007, DATE '2026-06-04', 175.90, 'Tarjeta', 109, 4);
INSERT INTO ventas VALUES (1008, DATE '2026-06-04', 320.00, 'Transferencia', 110, 5);
INSERT INTO ventas VALUES (1009, DATE '2026-06-05', 110.40, 'Efectivo', 111, 5);
INSERT INTO ventas VALUES (1010, DATE '2026-06-05', 89.30, 'Tarjeta', 112, 6);
INSERT INTO ventas VALUES (1011, DATE '2026-06-06', 56.80, 'Efectivo', 113, 7);
INSERT INTO ventas VALUES (1012, DATE '2026-06-06', 143.20, 'Tarjeta', 114, 8);
INSERT INTO ventas VALUES (1013, DATE '2026-06-07', 205.00, 'Transferencia', 103, 1);
INSERT INTO ventas VALUES (1014, DATE '2026-06-07', 77.70, 'Efectivo', 106, 2);
INSERT INTO ventas VALUES (1015, DATE '2026-06-08', 194.60, 'Tarjeta', 112, 6);
```

9.6. Ejercicios

Realizar las siguientes consultas:

1. Mostrar todos los empleados con el nombre de su tienda usando **INNER JOIN**.
2. Mostrar todos los empleados, aunque no tengan tienda asignada, usando **LEFT JOIN**.
3. Mostrar todas las tiendas, aunque no tengan empleados asignados, usando **RIGHT JOIN**.
4. Mostrar empleados sin tienda y tiendas sin empleados usando **FULL OUTER JOIN**.

5. Contar cuántas combinaciones genera un `CROSS JOIN` entre empleados y tiendas.
6. Mostrar cada empleado con el nombre de su supervisor usando `SELF JOIN`.
7. Mostrar solo los empleados cuyo supervisor sea `María López`.
8. Mostrar las tiendas que no tienen empleados asignados.
9. Mostrar los empleados que no tienen tienda asignada.
10. Mostrar las ventas con el nombre del empleado y el nombre de la tienda usando `INNER JOIN` entre las tres tablas.
11. Mostrar las ventas realizadas únicamente en tiendas de `Madrid`.
12. Mostrar el total de ventas realizadas por cada tienda.
13. Mostrar qué empleados han realizado ventas y en qué tienda las hicieron.
14. Mostrar todas las tiendas y, si existen, las ventas asociadas a cada una.
15. Mostrar los empleados que no han realizado ninguna venta.

10. Consultas útiles para revisar restricciones

10.1. Ver restricciones de las tablas

```
SELECT
    table_name,
    constraint_name,
    constraint_type,
    status
FROM user_constraints
WHERE table_name IN ('EMP_DEPARTAMENTOS', 'EMP_EMPLEADOS')
ORDER BY table_name, constraint_name;
```

Tipos de restricción más comunes:

Código	Significado
P	Primary key
R	Foreign key
U	Unique
C	Check o Not Null

10.2. Ver columnas de las tablas

```
SELECT
    table_name,
    column_name,
    data_type,
    nullable
FROM user_tab_columns
```

```
WHERE table_name IN ('EMP_DEPARTAMENTOS', 'EMP_EMPLEADOS')  
ORDER BY table_name, column_id;
```

Sesión 2

Consultas combinadas con JOIN

1. Contexto de la sesión

En esta sesión se trabajará con un caso práctico cotidiano: una aplicación de transporte tipo Uber.

La idea es representar una pequeña base de datos donde existen:

- clientes que solicitan viajes;
- conductores que realizan viajes;
- vehículos asignados a conductores;
- viajes realizados por clientes con conductores y vehículos;
- pagos asociados a los viajes.

El objetivo principal no es solo crear tablas, sino comprender cómo se relacionan entre sí y cómo consultar información repartida en varias tablas usando JOIN.

2. Esquema del caso práctico

El esquema estará formado por estas tablas:

```
UBR_CLIENTES  
UBR_CONDUCTORES  
UBR_VEHICULOS  
UBR_VIAJES  
UBR_PAGOS
```

3. Limpieza previa opcional

Antes de crear las tablas, si ya existen de una práctica anterior, se pueden borrar en este orden.

```
DROP TABLE ubr_pagos CASCADE CONSTRAINTS;  
DROP TABLE ubr_viajes CASCADE CONSTRAINTS;  
DROP TABLE ubr_vehiculos CASCADE CONSTRAINTS;  
DROP TABLE ubr_conductores CASCADE CONSTRAINTS;  
DROP TABLE ubr_clientes CASCADE CONSTRAINTS;
```

Si alguna tabla no existe, Oracle mostrará un error similar a:

```
ORA-00942: table or view does not exist
```

En ese caso no pasa nada. Significa que la tabla todavía no estaba creada.

4. Creación de tablas

4.1. Tabla **UBR_CLIENTES**

Esta tabla guarda los datos de los clientes de la aplicación.

```
CREATE TABLE ubr_clientes (  
    id_cliente NUMBER,  
    nombre VARCHAR2(80) NOT NULL,  
    email VARCHAR2(120) NOT NULL,  
    telefono VARCHAR2(20) NOT NULL,  
    ciudad VARCHAR2(60) NOT NULL,  
    fecha_alta DATE DEFAULT SYSDATE NOT NULL,  
    estado VARCHAR2(20) DEFAULT 'ACTIVO' NOT NULL,  
  
    CONSTRAINT pk_ubr_clientes  
        PRIMARY KEY (id_cliente),  
  
    CONSTRAINT uk_ubr_clientes_email  
        UNIQUE (email),  
  
    CONSTRAINT uk_ubr_clientes_telefono  
        UNIQUE (telefono),  
  
    CONSTRAINT ck_ubr_clientes_estado  
        CHECK (estado IN ('ACTIVO', 'INACTIVO'))  
);
```

Restricciones usadas:

- **PRIMARY KEY**: identifica de forma única a cada cliente.
- **NOT NULL**: obliga a que ciertos campos tengan valor.
- **UNIQUE**: evita repetir correos y teléfonos.
- **CHECK**: limita el estado a **ACTIVO** o **INACTIVO**.

4.2. Tabla **UBR_CONDUCTORES**

Esta tabla guarda los conductores registrados en la plataforma.

```
CREATE TABLE ubr_conductores (  
    id_conductor NUMBER,  
    nombre VARCHAR2(80) NOT NULL,
```

```

email VARCHAR2(120) NOT NULL,
telefono VARCHAR2(20) NOT NULL,
licencia VARCHAR2(30) NOT NULL,
ciudad VARCHAR2(60) NOT NULL,
calificacion NUMBER(2,1) DEFAULT 5 NOT NULL,
estado VARCHAR2(20) DEFAULT 'DISPONIBLE' NOT NULL,

CONSTRAINT pk_ubr_conductores
    PRIMARY KEY (id_conductor),

CONSTRAINT uk_ubr_conductores_email
    UNIQUE (email),

CONSTRAINT uk_ubr_conductores_telefono
    UNIQUE (telefono),

CONSTRAINT uk_ubr_conductores_licencia
    UNIQUE (licencia),

CONSTRAINT ck_ubr_conductores_calificacion
    CHECK (calificacion BETWEEN 1 AND 5),

CONSTRAINT ck_ubr_conductores_estado
    CHECK (estado IN ('DISPONIBLE', 'OCUPADO', 'INACTIVO'))
);

```

Restricciones usadas:

- Cada conductor tiene una clave primaria.
- El correo, teléfono y número de licencia no se pueden repetir.
- La calificación solo puede estar entre 1 y 5.
- El estado solo puede tomar valores definidos.

4.3. Tabla **UBR_VEHICULOS**

Esta tabla guarda los vehículos utilizados por los conductores.

```

CREATE TABLE ubr_vehiculos (
    id_vehiculo NUMBER,
    id_conductor NUMBER NOT NULL,
    matricula VARCHAR2(20) NOT NULL,
    marca VARCHAR2(50) NOT NULL,
    modelo VARCHAR2(50) NOT NULL,
    anio NUMBER(4) NOT NULL,
    color VARCHAR2(30) NOT NULL,
    tipo VARCHAR2(30) NOT NULL,
    estado VARCHAR2(20) DEFAULT 'ACTIVO' NOT NULL,
);

```

```

CONSTRAINT pk_ubr_vehiculos
    PRIMARY KEY (id_vehiculo),

CONSTRAINT fk_ubr_vehiculos_conductores
    FOREIGN KEY (id_conductor)
    REFERENCES ubr_conductores(id_conductor),

CONSTRAINT uk_ubr_vehiculos_matricula
    UNIQUE (matricula),

CONSTRAINT uk_ubr_vehiculos_conductor
    UNIQUE (id_conductor),

CONSTRAINT ck_ubr_vehiculos_anio
    CHECK (anio BETWEEN 2005 AND 2030),

CONSTRAINT ck_ubr_vehiculos_tipo
    CHECK (tipo IN ('ECONOMICO', 'CONFORT', 'PREMIUM', 'XL')),

CONSTRAINT ck_ubr_vehiculos_estado
    CHECK (estado IN ('ACTIVO', 'MANTENIMIENTO', 'INACTIVO'))
);

```

Puntos importantes:

- `id_conductor` es clave foránea porque cada vehículo pertenece a un conductor existente.
- `matricula` es única porque no puede haber dos vehículos con la misma matrícula.
- `id_conductor` también es único para simplificar el caso: un conductor tendrá un solo vehículo asignado en esta práctica.

4.4. Tabla `UBR_VIAJES`

Esta tabla guarda los viajes solicitados por los clientes.

```

CREATE TABLE ubr_viajes (
    id_viaje NUMBER,
    id_cliente NUMBER NOT NULL,
    id_conductor NUMBER NOT NULL,
    id_vehiculo NUMBER NOT NULL,
    origen VARCHAR2(120) NOT NULL,
    destino VARCHAR2(120) NOT NULL,
    fecha_viaje DATE NOT NULL,
    distancia_km NUMBER(6,2) NOT NULL,
    importe NUMBER(8,2) NOT NULL,
    estado VARCHAR2(20) DEFAULT 'COMPLETADO' NOT NULL,

    CONSTRAINT pk_ubr_viajes

```

```

PRIMARY KEY (id_viaje),

CONSTRAINT fk_ubr_viajes_clientes
FOREIGN KEY (id_cliente)
REFERENCES ubr_clientes(id_cliente),

CONSTRAINT fk_ubr_viajes_conductores
FOREIGN KEY (id_conductor)
REFERENCES ubr_conductores(id_conductor),

CONSTRAINT fk_ubr_viajes_vehiculos
FOREIGN KEY (id_vehiculo)
REFERENCES ubr_vehiculos(id_vehiculo),

CONSTRAINT ck_ubr_viajes_distancia
CHECK (distancia_km > 0),

CONSTRAINT ck_ubr_viajes_importe
CHECK (importe > 0),

CONSTRAINT ck_ubr_viajes_estado
CHECK (estado IN ('SOLICITADO', 'EN_CURSO', 'COMPLETADO', 'CANCELADO'))
);

```

Puntos importantes:

- Un viaje debe tener cliente, conductor y vehículo válidos.
- No se puede registrar un viaje con un cliente inexistente.
- No se puede registrar un viaje con un conductor inexistente.
- No se puede registrar un viaje con un vehículo inexistente.
- La distancia y el importe deben ser mayores que cero.

4.5. Tabla **UBR_PAGOS**

Esta tabla guarda el pago asociado a cada viaje.

```

CREATE TABLE ubr_pagos (
    id_pago NUMBER,
    id_viaje NUMBER NOT NULL,
    metodo_pago VARCHAR2(30) NOT NULL,
    importe_pagado NUMBER(8,2) NOT NULL,
    fecha_pago DATE NOT NULL,
    estado VARCHAR2(20) DEFAULT 'PAGADO' NOT NULL,

    CONSTRAINT pk_ubr_pagos
    PRIMARY KEY (id_pago),

```

```

CONSTRAINT fk_ubr_pagos_viajes
    FOREIGN KEY (id_viaje)
    REFERENCES ubr_viajes(id_viaje),

CONSTRAINT uk_ubr_pagos_viaje
    UNIQUE (id_viaje),

CONSTRAINT ck_ubr_pagos_metodo
    CHECK (metodo_pago IN ('TARJETA', 'EFECTIVO', 'PAYPAL', 'BIZUM')),

CONSTRAINT ck_ubr_pagos_importe
    CHECK (importe_pagado > 0),

CONSTRAINT ck_ubr_pagos_estado
    CHECK (estado IN ('PENDIENTE', 'PAGADO', 'RECHAZADO', 'DEVUELTO'))
);

```

Puntos importantes:

- Cada pago pertenece a un viaje existente.
- `id_viaje` es `UNIQUE` para que un viaje no tenga dos pagos en esta práctica.
- El método de pago queda limitado por un `CHECK`.

5. Inserción de registros

5.1. Insertar 20 clientes

```

INSERT INTO ubr_clientes VALUES (1, 'Ana Perez', 'ana.perez@mail.com', '600100001', 'Madrid', DATE '2025-01-05', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (2, 'Luis Gomez', 'luis.gomez@mail.com', '600100002', 'Madrid', DATE '2025-01-07', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (3, 'Maria Lopez', 'maria.lopez@mail.com', '600100003', 'Barcelona', DATE '2025-01-10', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (4, 'Carlos Ruiz', 'carlos.ruiz@mail.com', '600100004', 'Valencia', DATE '2025-01-12', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (5, 'Laura Martin', 'laura.martin@mail.com', '600100005', 'Sevilla', DATE '2025-01-15', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (6, 'Jorge Diaz', 'jorge.diaz@mail.com', '600100006', 'Madrid', DATE '2025-01-17', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (7, 'Sofia Torres', 'sofia.torres@mail.com', '600100007', 'Barcelona', DATE '2025-01-20', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (8, 'Pablo Romero', 'pablo.romero@mail.com', '600100008', 'Valencia', DATE '2025-01-22', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (9, 'Elena Castro', 'elena.castro@mail.com', '600100009', 'Sevilla', DATE '2025-01-25', 'ACTIVO');

```

```

INSERT INTO ubr_clientes VALUES (10, 'Miguel Navarro', 'miguel.navarro@mail.com', '600100010', 'Madrid', DATE '2025-01-27', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (11, 'Nuria Ortega', 'nuria.ortega@mail.com', '600100011', 'Barcelona', DATE '2025-02-01', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (12, 'David Molina', 'david.molina@mail.com', '600100012', 'Valencia', DATE '2025-02-03', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (13, 'Irene Vega', 'irene.vega@mail.com', '600100013', 'Sevilla', DATE '2025-02-05', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (14, 'Raul Sanchez', 'raul.sanchez@mail.com', '600100014', 'Madrid', DATE '2025-02-08', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (15, 'Patricia Leon', 'patricia.leon@mail.com', '600100015', 'Barcelona', DATE '2025-02-10', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (16, 'Alberto Ramos', 'alberto.ramos@mail.com', '600100016', 'Valencia', DATE '2025-02-12', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (17, 'Marta Flores', 'marta.flores@mail.com', '600100017', 'Sevilla', DATE '2025-02-15', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (18, 'Diego Santos', 'diego.santos@mail.com', '600100018', 'Madrid', DATE '2025-02-17', 'ACTIVO');
INSERT INTO ubr_clientes VALUES (19, 'Clara Gil', 'clara.gil@mail.com', '600100019', 'Barcelona', DATE '2025-02-19', 'INACTIVO');
INSERT INTO ubr_clientes VALUES (20, 'Hector Marin', 'hector.marin@mail.com', '600100020', 'Valencia', DATE '2025-02-21', 'ACTIVO');

```

5.2. Insertar 20 conductores

```

INSERT INTO ubr_conductores VALUES (1, 'Antonio Garcia', 'antonio.garcia@driver.com', '610200001', 'LIC-001', 'Madrid', 4.9, 'DISPONIBLE');
INSERT INTO ubr_conductores VALUES (2, 'Beatriz Alonso', 'beatriz.alonso@driver.com', '610200002', 'LIC-002', 'Madrid', 4.8, 'DISPONIBLE');
INSERT INTO ubr_conductores VALUES (3, 'Carmen Nieto', 'carmen.nieto@driver.com', '610200003', 'LIC-003', 'Barcelona', 4.7, 'DISPONIBLE');
INSERT INTO ubr_conductores VALUES (4, 'Daniel Prieto', 'daniel.prieto@driver.com', '610200004', 'LIC-004', 'Valencia', 4.6, 'OCUPADO');
INSERT INTO ubr_conductores VALUES (5, 'Eva Cano', 'eva.cano@driver.com', '610200005', 'LIC-005', 'Sevilla', 4.9, 'DISPONIBLE');
INSERT INTO ubr_conductores VALUES (6, 'Fernando Rojas', 'fernando.rojas@driver.com', '610200006', 'LIC-006', 'Madrid', 4.5, 'DISPONIBLE');
INSERT INTO ubr_conductores VALUES (7, 'Gloria Pascual', 'gloria.pascual@driver.com', '610200007', 'LIC-007', 'Barcelona', 4.4, 'DISPONIBLE');
INSERT INTO ubr_conductores VALUES (8, 'Hugo Vidal', 'hugo.vidal@driver.com', '610200008', 'LIC-008', 'Valencia', 4.3, 'OCUPADO');
INSERT INTO ubr_conductores VALUES (9, 'Isabel Luna', 'isabel.luna@driver.com', '610200009', 'LIC-009', 'Sevilla', 4.8, 'DISPONIBLE');
INSERT INTO ubr_conductores VALUES (10, 'Javier Mora', 'javier.mora@driver.com', '610200010', 'LIC-010', 'Madrid', 4.6, 'DISPONIBLE');
INSERT INTO ubr_conductores VALUES (11, 'Karla Reyes', 'karla.reyes@driver.com', '610200011', 'LIC-011', 'Barcelona', 4.7, 'DISPONIBLE');

```

```

INSERT INTO ubr_conductores VALUES (12, 'Leo Herrera', 'leo.herrera@driver.com', '610200012', 'LIC-012', 'Valencia', 4.5, 'DISPONIBLE');
INSERT INTO ubr_conductores VALUES (13, 'Monica Ibanez', 'monica.ibanez@driver.com', '610200013', 'LIC-013', 'Sevilla', 4.2, 'DISPONIBLE');
INSERT INTO ubr_conductores VALUES (14, 'Oscar Medina', 'oscar.medina@driver.com', '610200014', 'LIC-014', 'Madrid', 4.9, 'DISPONIBLE');
INSERT INTO ubr_conductores VALUES (15, 'Paula Arias', 'paula.arias@driver.com', '610200015', 'LIC-015', 'Barcelona', 4.4, 'DISPONIBLE');
INSERT INTO ubr_conductores VALUES (16, 'Quintin Soler', 'quintin.soler@driver.com', '610200016', 'LIC-016', 'Valencia', 4.1, 'INACTIVO');
INSERT INTO ubr_conductores VALUES (17, 'Rosa Campos', 'rosa.campos@driver.com', '610200017', 'LIC-017', 'Sevilla', 4.7, 'DISPONIBLE');
INSERT INTO ubr_conductores VALUES (18, 'Sergio Pardo', 'sergio.pardo@driver.com', '610200018', 'LIC-018', 'Madrid', 4.3, 'DISPONIBLE');
INSERT INTO ubr_conductores VALUES (19, 'Teresa Blasco', 'teresa.blasco@driver.com', '610200019', 'LIC-019', 'Barcelona', 4.6, 'DISPONIBLE');
INSERT INTO ubr_conductores VALUES (20, 'Victor Fuentes', 'victor.fuentes@driver.com', '610200020', 'LIC-020', 'Valencia', 4.5, 'DISPONIBLE');

```

5.3. Insertar 20 vehículos

```

INSERT INTO ubr_vehiculos VALUES (1, 1, 'UBR-1001', 'Toyota', 'Corolla', 2020, 'Blanco', 'ECONOMICO', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (2, 2, 'UBR-1002', 'Hyundai', 'Ioniq', 2021, 'Negro', 'CONFORT', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (3, 3, 'UBR-1003', 'Kia', 'Ceed', 2019, 'Gris', 'ECONOMICO', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (4, 4, 'UBR-1004', 'Tesla', 'Model 3', 2022, 'Rojo', 'PREMIUM', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (5, 5, 'UBR-1005', 'Seat', 'Leon', 2018, 'Azul', 'ECONOMICO', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (6, 6, 'UBR-1006', 'Renault', 'Megane', 2020, 'Blanco', 'CONFORT', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (7, 7, 'UBR-1007', 'Peugeot', '308', 2021, 'Negro', 'CONFORT', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (8, 8, 'UBR-1008', 'Mercedes', 'Clase E', 2022, 'Gris', 'PREMIUM', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (9, 9, 'UBR-1009', 'Skoda', 'Octavia', 2019, 'Blanco', 'ECONOMICO', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (10, 10, 'UBR-1010', 'Volkswagen', 'Passat', 2020, 'Azul', 'CONFORT', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (11, 11, 'UBR-1011', 'Nissan', 'Leaf', 2021, 'Verde', 'ECONOMICO', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (12, 12, 'UBR-1012', 'Ford', 'Focus', 2018, 'Rojo', 'ECONOMICO', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (13, 13, 'UBR-1013', 'Audi', 'A4', 2022, 'Negro', 'PREMIUM', 'ACTIVO');

```

```

INSERT INTO ubr_vehiculos VALUES (14, 14, 'UBR-1014', 'BMW', 'Serie 3', 20
21, 'Gris', 'PREMIUM', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (15, 15, 'UBR-1015', 'Citroen', 'C4', 202
0, 'Blanco', 'CONFORT', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (16, 16, 'UBR-1016', 'Opel', 'Astra', 201
7, 'Azul', 'ECONOMICO', 'INACTIVO');
INSERT INTO ubr_vehiculos VALUES (17, 17, 'UBR-1017', 'Dacia', 'Jogger', 2
022, 'Gris', 'XL', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (18, 18, 'UBR-1018', 'Toyota', 'Prius', 2
021, 'Blanco', 'ECONOMICO', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (19, 19, 'UBR-1019', 'Volvo', 'XC60', 202
3, 'Negro', 'XL', 'ACTIVO');
INSERT INTO ubr_vehiculos VALUES (20, 20, 'UBR-1020', 'Mazda', 'CX5', 202
0, 'Rojo', 'XL', 'MANTENIMIENTO');

```

5.4. Insertar 20 viajes

```

INSERT INTO ubr_viajes VALUES (1, 1, 1, 1, 'Atocha', 'Gran Via', DATE '202
5-03-01', 5.40, 12.50, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (2, 2, 2, 2, 'Sol', 'Chamartin', DATE '2025-
03-01', 8.20, 18.90, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (3, 3, 3, 3, 'Sants', 'Barceloneta', DATE '2
025-03-02', 6.10, 14.30, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (4, 4, 4, 4, 'Mestalla', 'Ruzafa', DATE '202
5-03-02', 4.30, 10.20, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (5, 5, 5, 5, 'Triana', 'Santa Justa', DATE
'2025-03-03', 7.50, 16.80, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (6, 6, 6, 6, 'Moncloa', 'Retiro', DATE '2025
-03-03', 6.90, 15.40, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (7, 7, 7, 7, 'Gracia', 'Diagonal', DATE '202
5-03-04', 3.80, 9.70, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (8, 8, 8, 8, 'Oceanografic', 'Centro', DATE
'2025-03-04', 9.60, 22.10, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (9, 9, 9, 9, 'Macarena', 'Nervion', DATE '20
25-03-05', 5.70, 13.60, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (10, 10, 10, 10, 'Barajas', 'Atocha', DATE
'2025-03-05', 17.90, 34.50, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (11, 11, 11, 11, 'Eixample', 'Sants', DATE
'2025-03-06', 4.90, 11.80, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (12, 12, 12, 12, 'Benimaclet', 'Malvarrosa',
DATE '2025-03-06', 6.40, 14.90, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (13, 13, 13, 13, 'Los Remedios', 'Alameda',
DATE '2025-03-07', 4.20, 10.00, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (14, 14, 14, 14, 'Lavapies', 'Salamanca', DA
TE '2025-03-07', 7.10, 17.20, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (15, 15, 15, 15, 'Camp Nou', 'Plaza Cataluny
a', DATE '2025-03-08', 8.80, 20.60, 'COMPLETADO');

```

```

INSERT INTO ubr_viajes VALUES (16, 1, 3, 3, 'Gran Via', 'Barajas', DATE '2025-03-08', 14.50, 29.90, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (17, 2, 4, 4, 'Chamartin', 'Sol', DATE '2025-03-09', 8.00, 18.20, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (18, 3, 5, 5, 'Barceloneta', 'Sagrada Familia', DATE '2025-03-09', 5.10, 12.40, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (19, 4, 6, 6, 'Ruzafa', 'Aeropuerto Valencia', DATE '2025-03-10', 11.30, 25.70, 'COMPLETADO');
INSERT INTO ubr_viajes VALUES (20, 5, 7, 7, 'Santa Justa', 'Triana', DATE '2025-03-10', 7.20, 16.10, 'COMPLETADO');

```

Observación:

- Los clientes 1 a 5 tienen más de un viaje.
- Los clientes 16 a 20 no tienen viajes en esta carga de datos.
- Los conductores 16 a 20 tienen vehículo, pero no tienen viajes registrados.
- Esto será útil para practicar `LEFT JOIN`.

5.5. Insertar 20 pagos

```

INSERT INTO ubr_pagos VALUES (1, 1, 'TARJETA', 12.50, DATE '2025-03-01', 'PAGADO');
INSERT INTO ubr_pagos VALUES (2, 2, 'TARJETA', 18.90, DATE '2025-03-01', 'PAGADO');
INSERT INTO ubr_pagos VALUES (3, 3, 'PAYPAL', 14.30, DATE '2025-03-02', 'PAGADO');
INSERT INTO ubr_pagos VALUES (4, 4, 'EFECTIVO', 10.20, DATE '2025-03-02', 'PAGADO');
INSERT INTO ubr_pagos VALUES (5, 5, 'BIZUM', 16.80, DATE '2025-03-03', 'PAGADO');
INSERT INTO ubr_pagos VALUES (6, 6, 'TARJETA', 15.40, DATE '2025-03-03', 'PAGADO');
INSERT INTO ubr_pagos VALUES (7, 7, 'PAYPAL', 9.70, DATE '2025-03-04', 'PAGADO');
INSERT INTO ubr_pagos VALUES (8, 8, 'TARJETA', 22.10, DATE '2025-03-04', 'PAGADO');
INSERT INTO ubr_pagos VALUES (9, 9, 'EFECTIVO', 13.60, DATE '2025-03-05', 'PAGADO');
INSERT INTO ubr_pagos VALUES (10, 10, 'TARJETA', 34.50, DATE '2025-03-05', 'PAGADO');
INSERT INTO ubr_pagos VALUES (11, 11, 'BIZUM', 11.80, DATE '2025-03-06', 'PAGADO');
INSERT INTO ubr_pagos VALUES (12, 12, 'PAYPAL', 14.90, DATE '2025-03-06', 'PAGADO');
INSERT INTO ubr_pagos VALUES (13, 13, 'TARJETA', 10.00, DATE '2025-03-07', 'PAGADO');
INSERT INTO ubr_pagos VALUES (14, 14, 'EFECTIVO', 17.20, DATE '2025-03-0

```

```

7', 'PAGADO');
INSERT INTO ubr_pagos VALUES (15, 15, 'TARJETA', 20.60, DATE '2025-03-08',
'PAGADO');
INSERT INTO ubr_pagos VALUES (16, 16, 'PAYPAL', 29.90, DATE '2025-03-08',
'PAGADO');
INSERT INTO ubr_pagos VALUES (17, 17, 'BIZUM', 18.20, DATE '2025-03-09',
'PAGADO');
INSERT INTO ubr_pagos VALUES (18, 18, 'TARJETA', 12.40, DATE '2025-03-09',
'PAGADO');
INSERT INTO ubr_pagos VALUES (19, 19, 'TARJETA', 25.70, DATE '2025-03-10',
'PAGADO');
INSERT INTO ubr_pagos VALUES (20, 20, 'EFECTIVO', 16.10, DATE '2025-03-1
0', 'PAGADO');

```

Confirmar los cambios:

```

COMMIT;

```

6. Comprobación de registros

Antes de trabajar los `JOIN`, conviene comprobar que todas las tablas tienen 20 registros.

```

SELECT COUNT(*) AS total_clientes
FROM ubr_clientes;

```

```

SELECT COUNT(*) AS total_conductores
FROM ubr_conductores;

```

```

SELECT COUNT(*) AS total_vehiculos
FROM ubr_vehiculos;

```

```

SELECT COUNT(*) AS total_viajes
FROM ubr_viajes;

```

```

SELECT COUNT(*) AS total_pagos
FROM ubr_pagos;

```

También se puede comprobar todo en una sola consulta:

```

SELECT 'CLIENTES' AS tabla, COUNT(*) AS total FROM ubr_clientes
UNION ALL
SELECT 'CONDUCTORES', COUNT(*) FROM ubr_conductores
UNION ALL
SELECT 'VEHICULOS', COUNT(*) FROM ubr_vehiculos

```

```
UNION ALL
SELECT 'VIAJES', COUNT(*) FROM ubr_viajes
UNION ALL
SELECT 'PAGOS', COUNT(*) FROM ubr_pagos;
```

7. Qué es un **JOIN**

Un **JOIN** permite consultar datos que están repartidos en varias tablas.

Por ejemplo, en la tabla **UBR_VIAJES** aparece esto:

```
ID_CLIENTE
ID_CONDUCTOR
ID_VEHICULO
```

Pero no aparece directamente el nombre del cliente, el nombre del conductor o la matrícula del vehículo.

Para obtener esa información completa, hay que combinar tablas.

Ejemplo conceptual:

```
UBR_VIAJES.ID_CLIENTE = UBR_CLIENTES.ID_CLIENTE
UBR_VIAJES.ID_CONDUCTOR = UBR_CONDUCTORES.ID_CONDUCTOR
UBR_VIAJES.ID_VEHICULO = UBR_VEHICULOS.ID_VEHICULO
```

8. Primer **INNER JOIN** : viajes con datos del cliente

```
SELECT
    v.id_viaje,
    c.nombre AS cliente,
    v.origen,
    v.destino,
    v.fecha_viaje,
    v.importe
FROM ubr_viajes v
INNER JOIN ubr_clientes c
    ON v.id_cliente = c.id_cliente;
```

Explicación:

```
FROM ubr_viajes v
```

La tabla principal será **UBR_VIAJES** y se le asigna el alias **v**.

```
INNER JOIN ubr_clientes c
```

Se combina con la tabla `UBR_CLIENTES`, usando el alias `c`.

```
ON v.id_cliente = c.id_cliente
```

Esta es la condición de unión. Oracle compara el identificador del cliente en la tabla de viajes con el identificador del cliente en la tabla de clientes.

Resultado esperado: se muestran los viajes junto con el nombre del cliente.

9. **INNER JOIN** con tres tablas

Ahora se mostrarán los viajes con cliente y conductor.

```
SELECT
    v.id_viaje,
    c.nombre AS cliente,
    d.nombre AS conductor,
    v.origen,
    v.destino,
    v.importe
FROM ubr_viajes v
INNER JOIN ubr_clientes c
    ON v.id_cliente = c.id_cliente
INNER JOIN ubr_conductores d
    ON v.id_conductor = d.id_conductor;
```

Aquí se están uniendo tres tablas:

```
UBR_VIAJES + UBR_CLIENTES + UBR_CONDUCTORES
```

La tabla `UBR_VIAJES` funciona como tabla central, porque tiene las claves foráneas hacia clientes y conductores.

10. **INNER JOIN** con cuatro tablas

Ahora se añade la tabla de vehículos.

```
SELECT
    v.id_viaje,
    c.nombre AS cliente,
    d.nombre AS conductor,
    vh.marca,
    vh.modelo,
    vh.matricula,
    v.origen,
    v.destino,
    v.importe
```

```

FROM ubr_viajes v
INNER JOIN ubr_clientes c
    ON v.id_cliente = c.id_cliente
INNER JOIN ubr_conductores d
    ON v.id_conductor = d.id_conductor
INNER JOIN ubr_vehiculos vh
    ON v.id_vehiculo = vh.id_vehiculo;

```

Esta consulta permite responder preguntas como:

```

¿Qué cliente viajó?
¿Quién fue el conductor?
¿Qué vehículo se utilizó?
¿Desde dónde salió?
¿A dónde llegó?
¿Cuánto costó?

```

11. INNER JOIN con cinco tablas

Ahora se añade la tabla de pagos.

```

SELECT
    v.id_viaje,
    c.nombre AS cliente,
    d.nombre AS conductor,
    vh.matricula,
    v.origen,
    v.destino,
    v.importe AS importe_viaje,
    p.metodo_pago,
    p.importe_pagado,
    p.estado AS estado_pago
FROM ubr_viajes v
INNER JOIN ubr_clientes c
    ON v.id_cliente = c.id_cliente
INNER JOIN ubr_conductores d
    ON v.id_conductor = d.id_conductor
INNER JOIN ubr_vehiculos vh
    ON v.id_vehiculo = vh.id_vehiculo
INNER JOIN ubr_pagos p
    ON v.id_viaje = p.id_viaje;

```

Esta consulta muestra el ciclo completo:

```

Cliente -> Viaje -> Conductor -> Vehículo -> Pago

```

12. Filtros sobre consultas con JOIN

Un `JOIN` también puede usar `WHERE` para filtrar resultados.

12.1. Viajes de un cliente concreto

```
SELECT
    c.nombre AS cliente,
    v.origen,
    v.destino,
    v.importe
FROM ubr_clientes c
INNER JOIN ubr_viajes v
    ON c.id_cliente = v.id_cliente
WHERE c.nombre = 'Ana Perez';
```

12.2. Viajes superiores a 20 euros

```
SELECT
    v.id_viaje,
    c.nombre AS cliente,
    d.nombre AS conductor,
    v.importe
FROM ubr_viajes v
INNER JOIN ubr_clientes c
    ON v.id_cliente = c.id_cliente
INNER JOIN ubr_conductores d
    ON v.id_conductor = d.id_conductor
WHERE v.importe > 20;
```

12.3. Viajes pagados con tarjeta

```
SELECT
    v.id_viaje,
    c.nombre AS cliente,
    v.importe,
    p.metodo_pago
FROM ubr_viajes v
INNER JOIN ubr_clientes c
    ON v.id_cliente = c.id_cliente
INNER JOIN ubr_pagos p
    ON v.id_viaje = p.id_viaje
WHERE p.metodo_pago = 'TARJETA';
```

13. Uso de alias en consultas con JOIN

Los alias hacen que las consultas sean más cortas y legibles.

Sin alias:

```
SELECT
    ubr_viajes.id_viaje,
    ubr_clientes.nombre,
    ubr_viajes.origen,
    ubr_viajes.destino
FROM ubr_viajes
INNER JOIN ubr_clientes
    ON ubr_viajes.id_cliente = ubr_clientes.id_cliente;
```

Con alias:

```
SELECT
    v.id_viaje,
    c.nombre,
    v.origen,
    v.destino
FROM ubr_viajes v
INNER JOIN ubr_clientes c
    ON v.id_cliente = c.id_cliente;
```

La segunda versión es más cómoda para trabajar.

14. Diferencia entre INNER JOIN y LEFT JOIN

14.1. INNER JOIN

El **INNER JOIN** muestra solo los registros que tienen coincidencia en ambas tablas.

Ejemplo:

```
SELECT
    c.id_cliente,
    c.nombre,
    v.id_viaje,
    v.importe
FROM ubr_clientes c
INNER JOIN ubr_viajes v
    ON c.id_cliente = v.id_cliente;
```

Esta consulta mostrará solo clientes que tienen viajes.

14.2. LEFT JOIN

El **LEFT JOIN** muestra todos los registros de la tabla izquierda, aunque no tengan coincidencia en la tabla derecha.

```

SELECT
    c.id_cliente,
    c.nombre,
    v.id_viaje,
    v.importe
FROM ubr_clientes c
LEFT JOIN ubr_viajes v
    ON c.id_cliente = v.id_cliente;

```

Esta consulta mostrará:

- clientes con viajes;
- clientes sin viajes.

Cuando un cliente no tenga viajes, las columnas de `UBR_VIAJES` aparecerán como `NULL`.

15. Buscar clientes sin viajes

Para encontrar clientes que no han realizado ningún viaje:

```

SELECT
    c.id_cliente,
    c.nombre,
    c.email,
    c.ciudad
FROM ubr_clientes c
LEFT JOIN ubr_viajes v
    ON c.id_cliente = v.id_cliente
WHERE v.id_viaje IS NULL;

```

Explicación:

- Primero se muestran todos los clientes con `LEFT JOIN`.
- Luego se filtran los casos donde no existe viaje.
- `v.id_viaje IS NULL` significa que no se encontró ningún viaje asociado.

16. Buscar conductores sin viajes

```

SELECT
    d.id_conductor,
    d.nombre,
    d.ciudad,
    d.estado
FROM ubr_conductores d
LEFT JOIN ubr_viajes v
    ON d.id_conductor = v.id_conductor
WHERE v.id_viaje IS NULL;

```

Esta consulta permite detectar conductores registrados que todavía no tienen viajes realizados.

17. Buscar vehículos sin viajes

```
SELECT
    vh.id_vehiculo,
    vh.matricula,
    vh.marca,
    vh.modelo,
    vh.estado
FROM ubr_vehiculos vh
LEFT JOIN ubr_viajes v
    ON vh.id_vehiculo = v.id_vehiculo
WHERE v.id_viaje IS NULL;
```

Esta consulta muestra vehículos que existen en la plataforma, pero todavía no aparecen en ningún viaje.

18. Consultas para practicar

18.1. Nivel básico

1. Mostrar todos los viajes con el nombre del cliente.
2. Mostrar todos los viajes con el nombre del conductor.
3. Mostrar todos los viajes con la matrícula del vehículo.
4. Mostrar todos los pagos con el origen y destino del viaje.
5. Mostrar los viajes cuyo importe sea mayor que 15 euros.
6. Mostrar los viajes pagados con **TARJETA**.
7. Mostrar los viajes realizados por clientes de **Madrid**.
8. Mostrar los conductores que tienen vehículos **PREMIUM**.
9. Mostrar los vehículos y el nombre de su conductor.
10. Mostrar los clientes que han realizado algún viaje.

18.2. Nivel intermedio

1. Mostrar cliente, conductor, vehículo, origen, destino, importe y método de pago.
2. Mostrar los viajes realizados con vehículos de tipo **PREMIUM**.
3. Mostrar los viajes donde el conductor tenga calificación mayor o igual a 4.7.
4. Mostrar clientes que no tienen viajes registrados.
5. Mostrar conductores que no tienen viajes registrados.
6. Mostrar vehículos que no han sido usados en viajes.
7. Mostrar viajes cuyo importe del viaje coincida con el importe pagado.

8. Mostrar viajes de clientes de **Barcelona** pagados con **PAYPAL** .
9. Mostrar viajes realizados en vehículos de color **Blanco** .
10. Mostrar los viajes con distancia superior a 8 kilómetros y pago realizado por **TARJETA** .

19. Ejemplo de consulta final

```
SELECT
    c.nombre AS cliente,
    c.ciudad AS ciudad_cliente,
    d.nombre AS conductor,
    d.calificacion,
    vh.marca,
    vh.modelo,
    vh.tipo,
    v.origen,
    v.destino,
    v.distancia_km,
    v.importe,
    p.metodo_pago,
    p.estado AS estado_pago
FROM ubr_viajes v
INNER JOIN ubr_clientes c
    ON v.id_cliente = c.id_cliente
INNER JOIN ubr_conductores d
    ON v.id_conductor = d.id_conductor
INNER JOIN ubr_vehiculos vh
    ON v.id_vehiculo = vh.id_vehiculo
INNER JOIN ubr_pagos p
    ON v.id_viaje = p.id_viaje
WHERE v.importe > 15
    AND p.estado = 'PAGADO';
```

Esta consulta combina cinco tablas y filtra solo viajes pagados con importe superior a 15 euros.

20. Entregables de esta clase

- Solo debes entregar para cada caso de uso el script .sql. Sería dos .sql, el de la sesión 1 y la sesión 2